Mathematics in CP

Arman Ferdous

Sections

- Series and sums
- Ceiling and Floor functions
- Divisors and divisibility
- Prime Numbers, detection and generation
- Modular arithmetic
- Binary Exponentiation
- Extended Euclidean Algorithm

Sums and Series

- https://en.wikipedia.org/wiki/Summation#General_identities
- Series formulas

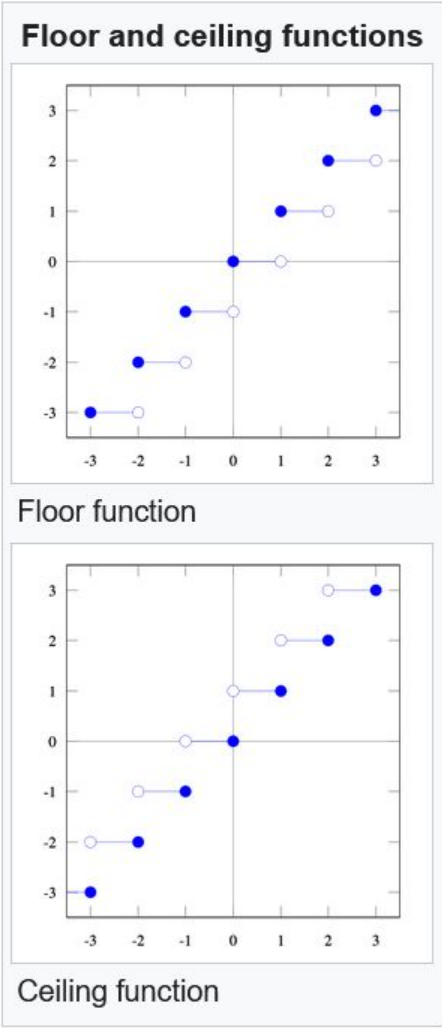
https://codeforces.com/contest/1853/problem/B

Ceiling and Floor Functions

- ceil(x): Smallest integer >= x
- floor(x): Largest integer <= x
- x - 1 < floor(x) <= x <= ceil(x) < x + 1
  - Because in a half open interval of length 1, there is exactly 1 integer.
- ceil(x) = - floor(-x)
- When m is positive:

$$\left\lceil \frac{n}{m} \right\rceil = \left\lfloor \frac{n+m-1}{m} \right\rfloor = \left\lfloor \frac{n-1}{m} \right\rfloor + 1,$$

- Remainder $$x \bmod y = x - y \left\lfloor \frac{x}{y} \right\rfloor$$



https://codeforces.com/problemset/problem/616/E

Divisors and divisibility

- If a number D divides X (leaves remainder = 0), we write:

  D | X (read as “D divides X”)

  6 | 12
- D | X also means, that there exists an integer k such that D * k = X.
- If D | X, then D is a divisor/factor of X. And X is a multiple of D.
- Often, we may need to find all the divisors of a number.

  12 = 1 * 12 = 2 * 6 = 3 * 4
- Divisors come in pairs. If d | x, then (x/d) | x too.
- How to find all divisors of a number quickly?
- How to find all divisors of all the numbers from 1 to n fast?

```
vector<vector<int>> divisors;

int main () {
    int n; cin >> n;
    divisors.resize(n + 1);

    for (int i = 1; i <= n; ++i)
        for (int j = i; j <= n; j += i)
            divisors[j].push_back(i);

}
```

Divisibility properties

- https://math.libretexts.org/Bookshelves/Combinatorics_and_Discrete_Mathematics/Elementary_Number_Theory_(Barrus_and_Clark)/01%3A_Chapters/1.04%3A_Elementary_Divisibility_Properties

Prime Numbers

- A positive integer p which is only divisible by 1 and p is called a prime.
- Why care about primes? Fundamental Theorem of Arithmetic

Primality Testing

- How to check if a given number is a prime?
-

```
bool isPrime (ll x) {
    if (x < 2) return false;
    for (ll i = 2; i * i <= x; ++i)
        if (x % i == 0)
            return false;
    return true;
}
```

Prime Generation

- How to generate all prime numbers upto n?
- Sieve of Eratosthenes

```
vector<bool> isc; // is composite
vector<int> pr;   // final prime list

void sieve (int n) {
    for (int i = 2; i * 1LL * i <= n; ++i) if (!isc[i]) {
        for (int j = i * i; j <= n; j += i)
            isc[j] = true;
    }
    for (int i = 2; i <= n; ++i) if (!isc[i])
        pr.push_back(i);
}
```

- Complexity: O(n ln ln n)

https://codeforces.com/problemset/problem/26/A

https://toph.co/p/n-th-prime

https://www.spoj.com/problems/VECTAR8/

Prime Factorization

- If we are given a number x, how can we prime factorize it?
- Method 1: A O(sqrt(x)) approach, suitable upto 1e16.

```
vector<pair<ll, int>> prime_factorize (ll n) {
    vector<pair<ll, int>> res;
    for (ll i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            int exp = 0;
            while (n % i == 0) {
                n /= i;
                exp++;
            }
            res.emplace_back(i, exp);
        }
    }
    if (n > 1) res.emplace_back(n, 1);
    return res;
}
```

Prime Factorization

```
vector<pair<ll,int>> factorize (int n) {
    vector<pair<ll,int>> res;
    while (n > 1) {
        int p = spf[n];
        int exp = 0;
        while (n % p == 0) n /= p, exp++;
        res.emplace_back(p, exp);
    }
    return res;
}
```

https://codeforces.com/problemset/problem/2125/C

https://codeforces.com/gym/106178/problem/F

NOD and SOD

- Suppose after prime factorizing n you get:

$$n = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k}$$

- Number of Divisors of n

$$NOD = (e_1 + 1)(e_2 + 1) \dots (e_k + 1)$$

- Sum of Divisors of n

$$SOD = \frac{p_1^{e_1+1} - 1}{p_1 - 1} \times \dots \times \frac{p_k^{e_k+1} - 1}{p_k - 1}$$

https://cses.fi/problemset/task/1713

GCD and LCM

- GCD(a, b) = largest integer x, that divides both a and b.

  1 <= GCD(a, b) <= min(a, b)
- LCM(a, b) = smallest integer y, that is a multiple of both a and b.

  a * b >= LCM(a, b) >= max(a, b)
- FACT: GCD(a, b) * LCM(a, b) = a * b. Why?

Euclidean Algorithm

- Properties of GCD:
- If a > b, GCD(a, b) = GCD(a - b, b)
- Extend this to obtain GCD(a, b) = GCD(b, a % b)

https://codeforces.com/contest/1916/problem/B

https://codeforces.com/contest/2226/problem/B

Equivalence Relation

- Suppose you have a set S. You define a relation ~ on some pair of elements. If x is related to y, then x ~ y.
- Now the relation is an equivalence relation iff for every x, y, z in S:
  - Reflexivity: x ~ x.
  - Symmetry: If x ~ y then y ~ x.
  - Transitivity: If x ~ y and y ~ z then x ~ z.
- Examples of equivalence relations:
  - Two matrices are related if they have the same determinant.
  - Two integers are related if they have the same parity.
  - Two triangles are related if they are similar.
- False examples:
  - For two integers, A is related to B if A <= B.
  - Two integers (>1) are related if they have gcd > 1.
  - You and someone are related if you are friends.

Equivalence Relations create a Partition of the Set

- An equivalence relation on S, partitions the members of S into sets P_1, P_2, … where each set is disjoint from one another and every member of S is present in exactly one of P_i.
- Now, we will partition the set of integers using the following equivalence relation

  For a fixed M, we write X ~ Y if M | (X - Y)
- Prove that this is an equivalence relation.

Modular Arithmetic

- $X \sim Y$ if $M \mid (X - Y)$
- $X \sim Y$ if $X - Y = M * k$
- $X \sim Y$ if $X \equiv Y \pmod{M}$
- How do the partitions look like?
- $[0] = \{..., -M, 0, M, 2M, \dots\}$
- $[1] = \{..., 1-M, 1, 1+M, 1+2M, \dots\}$
- …
- $[M - 1] = \{... -1, M-1, 2M-1, \dots\}$
- The values from $[0, M-1]$ are the principal values.
- Principal values can be found by taking the remainder when dividing by M.

Properties of Modular Arithmetic

- https://en.wikipedia.org/wiki/Modular_arithmetic#Basic_properties

Binary Exponentiation

- How to calculate a^b % m?

```
// calculate a^b (mod m)
int powmod (int a, int b, int M) {
    int res = 1;
    while (b) {
        int last_bit = b % 2;
        if (last_bit == 1) {
            res = (res * a) % M;
        }
        b /= 2;
        a = (a * a) % M;
    }
    return res;
}
```

Problems

- Find the least positive integer x such that

  $$71 \equiv x + 5 \pmod 8$$

- Find the last digit of 9^1329.
- Divisibility hacks (2, 3, 4, 5, 6, 7, 8, 9)
- Find the least positive integer x such that

  $$23 \equiv 4x \pmod 8$$

Bézout’s Identity

- https://en.wikipedia.org/wiki/B%C3%A9zout%27s_identity
- For any two integers a and b, there exists positive integers x and y such that

$$ax + by = \gcd(a, b)$$

Extended Euclidean Algorithm

- A way to fish out those x and y values of Bezout.
- Run the normal Euclidean algorithm, but figure out the values of x and y from base cases.
- Suppose you found x_1, and y_1 for GCD(b, a % b).

  Then, $b * x_1 + (a \% b) * y_1 = g$
- Now, we want x and y for:

  $a * x + b * y = g$
- Write a % b = a - floor(a / b) * b and substitute in the first equation.

Linear Diophantine Equations

- Equations of the form:

$$Ax + By = C$$

- How to solve this for integer values only? Geometric representation?

Modular Multiplicative Inverse

- Now, if we want to find a way to solve $23 \equiv 4x \pmod 8$, by dividing by 4 on both sides, what we really want to find is a number y to multiply on both sides, such that the 4 on the RHS cancels out.
- Modular Multiplicative Inverse: of a number x under modulo M is a number y such that $x * y = 1 \pmod M$
- So, $x * y - 1 = M * k$
- Which gives, $x * y + M * (-k) = 1$
- We know x and M, we solve for y and (-k) using egcd and use y as the inverse!
- So when do inverses exist?

Fermat's Little Theorem

- For any prime number p, if p does not divide a then

$$a^{p-1} \equiv 1 \pmod{p}$$

  How?
- Take S = {1, 2, …, p - 1}.
- Now let aS = {1a, 2a, …, (p-1)a} all modulo p
- Show that aS is just a permutation of S.
- Then, 1 * 2 * … * (p-1) ≡ (1a) * (2a) * … * ((p-1)*a) (mod p)

Why do we need FLT?

- FLT can help reduce large powers into smaller values.
- If we need to calculate a^b % p, where b is too large, we can't simply calculate a^(b%p) % p, this would be incorrect.
- Using FLT, we can raise our modulo towards the exponent!
- Example: evaluate

$$15^{8^{13}} \pmod{11}$$

Euler’s Totient Function

- $\varphi(n)$ = number of integers < n which are coprime with n.
- Simply, $\varphi(n)$ is the number of elements modulo n, which have inverses.
- How to calculate $\varphi(n)$?
-

Euler’s Theorem

- If a, n are coprime (i.e. gcd(a, n) = 1), then

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$